

利用適用於 Android 的質感設計動態效果建構精美的轉換效果

來源：<https://codelabs.developers.google.com/codelabs/material-motion-android?hl=zh-tw>

步驟數：9

使用 Android 程式庫和 Kotlin 的 Material 元件的轉換，將 Material 的動態系統加入 Reply 應用程式。

目錄

1. [1. 簡介](#)
2. [2. 設定開發環境](#)
3. [3. 熟悉範例應用程式程式碼](#)
4. [4. 從電子郵件名單新增容器轉換轉場至電子郵件詳細資料頁面](#)
5. [5. 從 FAB 新增容器轉換轉場效果至撰寫電子郵件頁面](#)
6. [6. 從搜尋圖示到搜尋檢視畫面頁面，新增共用 Z 軸轉換](#)
7. [7. 在信箱頁面之間新增「淡出淡入」轉場效果](#)
8. [8. 從電子郵件地址方塊到資訊卡檢視畫面，新增容器轉換轉場效果](#)
9. [9. 大功告成](#)

1. 簡介

Material Design 是一套系統，可協助您打造大膽且美觀的數位產品。只要在一致的原則和元件下整合樣式、品牌、互動和動態效果，產品團隊就能發揮最大的設計潛力。



開發人員可透過 Material 元件 (MDC) 實作 Material Design。MDC 由 Google 的工程師和 UX 設計師團隊打造，提供數十種美觀實用的 UI 元件，適用於 Android、iOS、網頁和 Flutter。material.io/develop

什麼是 Android 適用的 Material 動作系統？

Android 適用的 **Material 動態效果系統** 是 MDC-Android 程式庫中的一組轉場效果模式，可協助使用者瞭解及瀏覽應用程式，詳情請參閱 [Material Design 指南](#)。

Material 轉換主要有四種模式，分別是：

- **容器轉換**：轉換含有容器的 UI 元素，並將一個元素無縫轉換為另一個元素，在兩個不同的 UI 元素之間建立可見的連結。
- **共用軸**：轉換 UI 元素，這些 UI 元素彼此間具有空間或導覽的關係；轉換時共用 X、Y 或 Z 軸，藉此強調元素間的關係。
- **淡出淡入**：轉換彼此間沒有緊密關係的 UI 元素；使用連續淡出和淡入效果，並縮放進入的元素。
- **淡化**：用於進入或退出畫面範圍的 UI 元素。

MDC-Android 程式庫提供這些模式的轉場效果類別，以 [AndroidX Transition 程式庫](#) (`androidx.transition`) 和 [Android Transition Framework](#) (`android.transition`) 為基礎：

AndroidX

- 適用於 `com.google.android.material.transition` 套件
- 支援 API 級別 14 以上版本
- 支援片段和檢視區塊，但不支援活動或視窗
- 包含回溯移植的錯誤修正，以及各 API 級別的一致行為

架構

- 適用於 `com.google.android.material.transition.platform` 套件
- 支援 API 級別 21 以上版本
- 支援片段、檢視區塊、活動和視窗
- 錯誤修正不會回溯移植，且不同 API 級別的行為可能有所不同

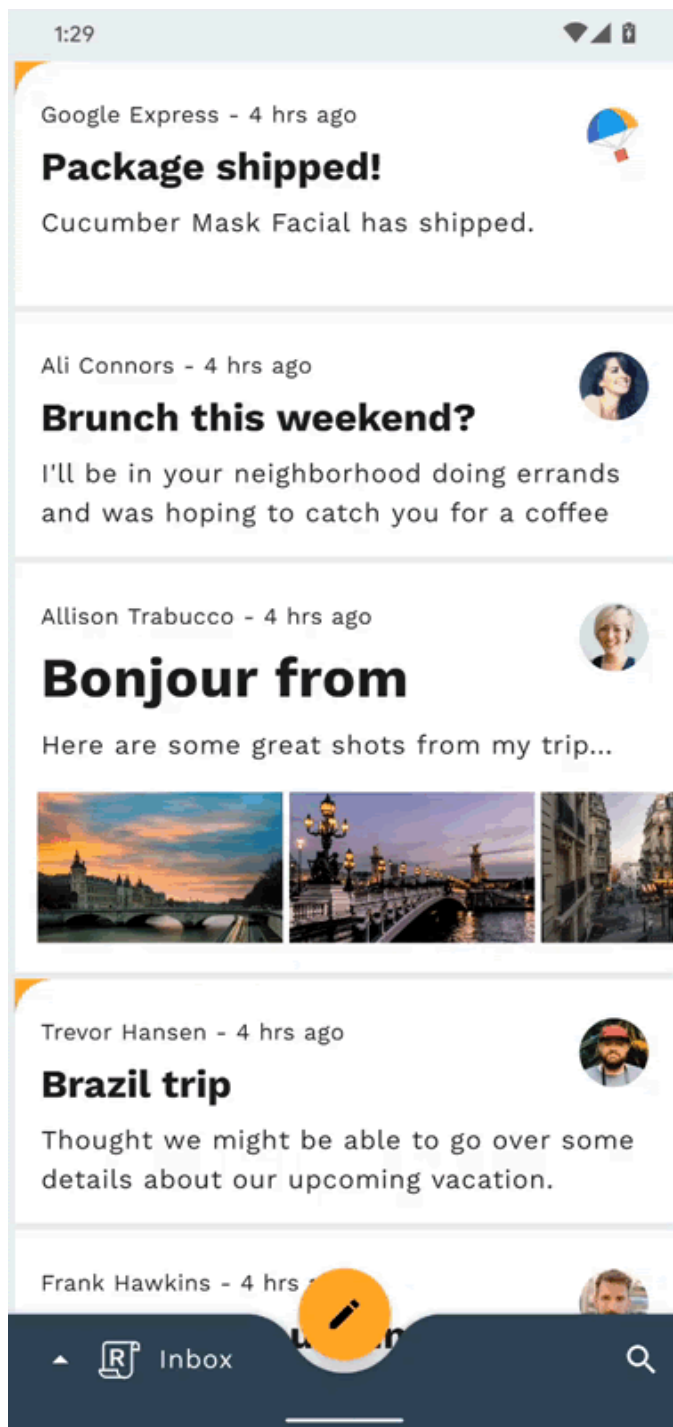
在本程式碼研究室中，您將使用以 AndroidX 程式庫為基礎建構的 Material 轉換效果，也就是主要著重於 Fragment 和 View。

建構項目

本程式碼研究室將引導您使用 Kotlin，在名為「Reply」的 Android 電子郵件應用程式範例中建構一些轉場效果，示範如何使用 MDC-Android 程式庫的轉場效果，自訂應用程式的外觀和風格。

我們會提供 Reply 應用程式的範例程式碼，您將在應用程式中加入下列 Material 轉場效果，如下方完成的程式碼研究室 GIF 所示：

- 從電子郵件名單到電子郵件詳細資料頁面的**容器轉換**轉場效果
- **容器轉換**：從懸浮動作按鈕 (FAB) 轉換至撰寫電子郵件頁面
- **共用 Z 軸**：從搜尋圖示到搜尋檢視頁面的轉場效果
- 信箱頁面之間的**淡出**轉場效果
- **容器轉換**：從電子郵件地址方塊轉換為資訊卡檢視



在本程式碼研究室中，您將使用 MDC-Android 動態效果系統提供的轉場效果。如要進一步瞭解 MDC-Android 主題設定和元件系統，請參閱[入門指南](#)。

軟硬體需求

- 具備 Android 開發和 Kotlin 的基本知識
 - **Android Studio** (如果沒有，請[在這裡](#)下載)
 - Android 模擬器或裝置 (可透過 Android Studio 取得)
 - 程式碼範例 (請參閱下一個步驟)
-

2. 設定開發環境

啟動 Android Studio

開啟 **Android Studio** 時，應該會顯示名為「Welcome to Android Studio」的視窗。不過，如果您是第一次啟動 Android Studio，請使用預設值完成 **Android Studio** 設定精靈步驟。下載及安裝必要檔案可能需要幾分鐘的時間，因此您可以讓這項作業在背景執行，同時進行下一節的步驟。

選項 1：從 GitHub 複製入門程式碼研究室應用程式

如要從 GitHub 複製這個[程式碼研究室](#)，請執行下列指令：

```
git clone https://github.com/material-components/material-components-android-motion-codelab.git
cd material-components-android-motion-codelab
```

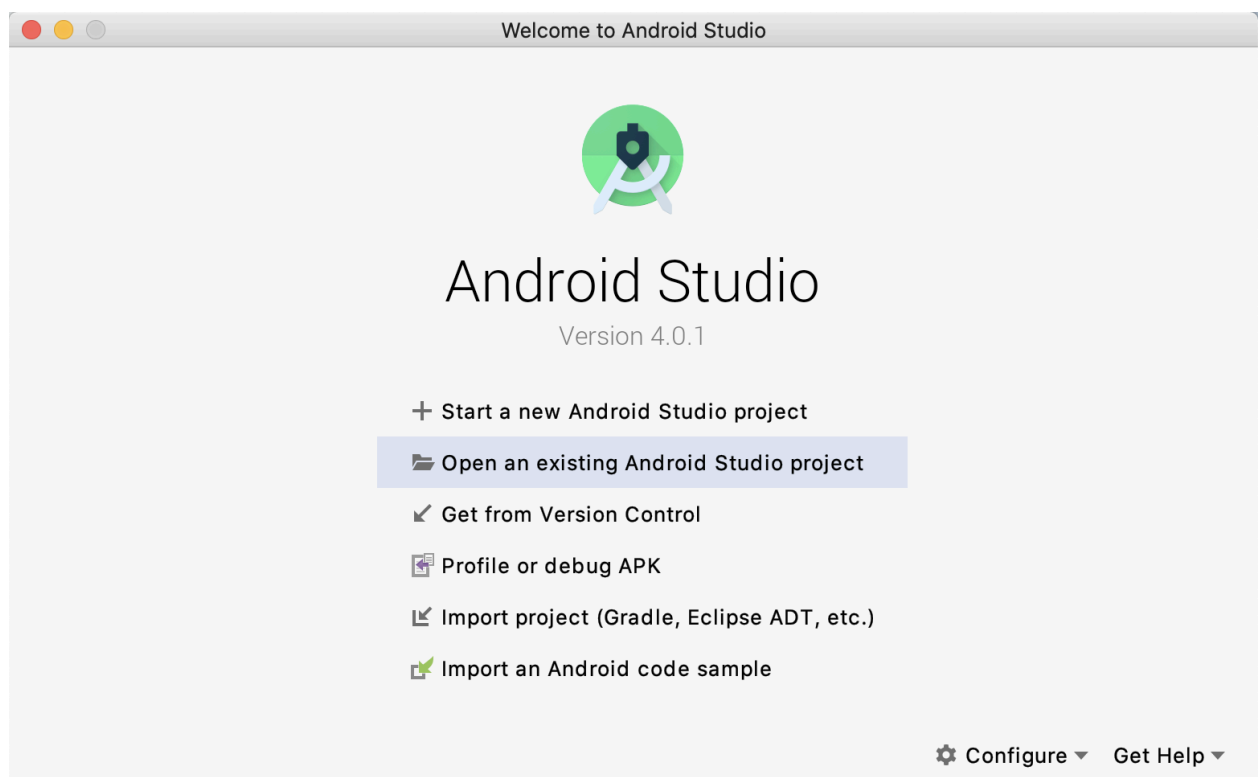
如需更多說明，請參閱「[從 GitHub 複製存放區](#)」。

選項 2：下載 入門程式碼研究室應用程式的 ZIP 檔案

範例應用程式位於 `material-components-android-motion-codelab-develop` 目錄中。

在 Android Studio 中載入範例程式碼

1. 設定精靈完成後，系統會顯示「Welcome to Android Studio」視窗，請按一下「Open an existing Android Studio project」。



2. 前往安裝程式碼範例的目錄，然後選取範例目錄來開啟專案。
3. 稍待片刻，Android Studio 會建構及同步處理專案，Android Studio 視窗底部的活動指標會顯示進度。

如果 Android Studio 提示您更新專案的 Android Gradle 外掛程式版本，請忽略該提示，因為專案可能未經過最新 Gradle 版本測試，更新可能會導致當機或其他編譯問題。

4. 此時，Android Studio 可能會引發一些建構錯誤，因為您缺少 Android SDK 或建構工具，如下所示。按照 Android Studio 中的操作說明安裝/更新這些項目，然後[同步處理](#)專案。如果仍有問題，請按照[指南](#)，使用 SDK Manager 更新工具。

```
Failed to find target with hash string 'android-28' in: /Users/username/Library/Android/sdk
Install missing plaform(s) and sync
```

驗證專案依附元件

專案需要 **MDC-Android** 程式庫的依附元件。您下載的程式碼範例應該已列出這項依附元件，但我們還是來看看設定，確保一切正常。

前往 `app` 模組的 `build.gradle` 檔案，確認 `dependencies` 區塊包含 MDC-Android 的依附元件：

```
implementation 'com.google.android.material:material:1.2.0'
```

執行範例應用程式

1. 確認裝置選項左側的建構設定為 `app`。
2. 按下綠色的「Run / Play」按鈕，即可建構及執行應用程式。

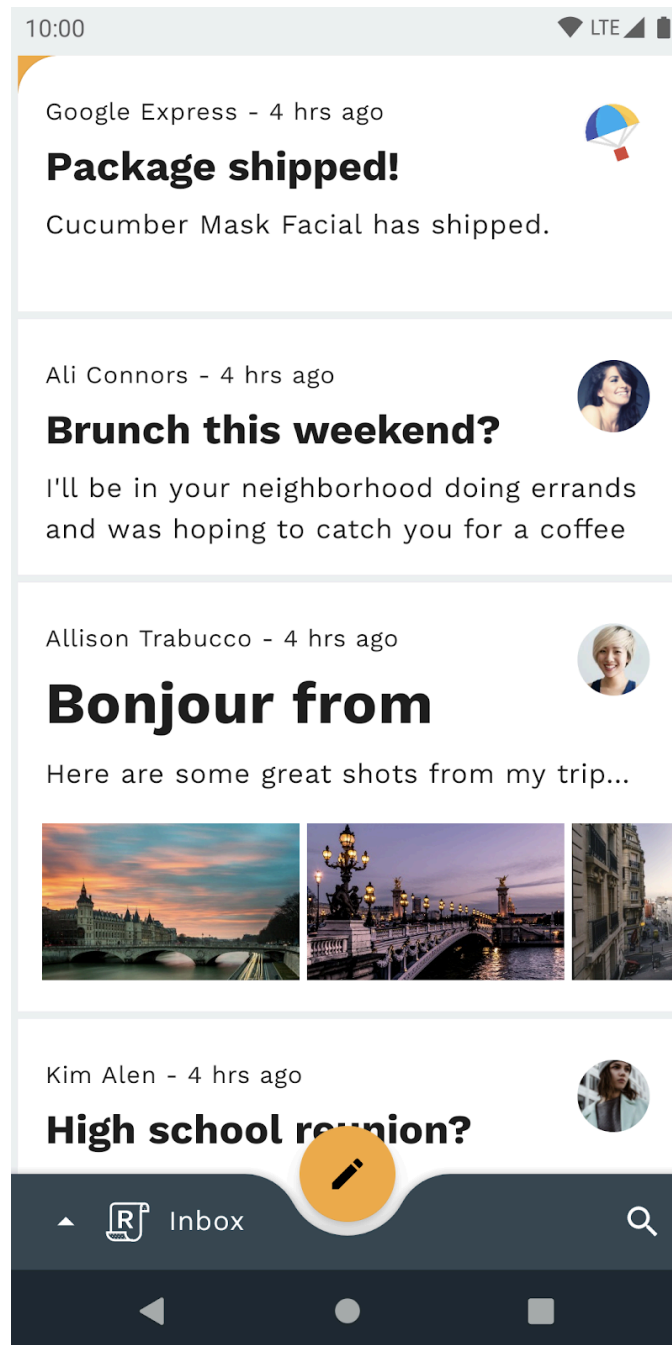


3. 在「Select Deployment Target」視窗中，如果可用裝置清單中已有 **Android** 裝置，請跳至步驟 8。如果沒有，請按一下「Create New Virtual Device」。
4. 在「Select Hardware」畫面中，選取手機裝置 (例如 **Pixel 3**)，然後按一下「Next」。
5. 在「系統映像檔」畫面中，選取**最新的 Android 版本**，最好是最高的 API 級別。如果尚未安裝，請按一下顯示的「下載」連結，然後完成下載。
6. 點選 [下一步]。
7. 在「Android 虛擬裝置 (AVD)」畫面上，保留預設設定並點選「Finish」。
8. 從部署目標對話方塊中選取 **Android 裝置**。
9. 按一下 [確定]。
10. Android Studio 會建構及部署應用程式，並在目標裝置上自動開啟。

如果在建構時收到 `INSTALL_FAILED_INVALID_APK` 錯誤，請嘗試在 Android Studio 中依序選取「Build」(建構) -> 「Clean Project」(清除專案)。

如果無法新增依附元件或順利執行應用程式，請停止並排解開發人員環境的問題。如要進一步瞭解如何建構及執行應用程式，請參閱 [Android Studio 指南](#)。

太棒了，Reply 首頁的範例程式碼應在模擬器中執行。這時應該會看到收件匣，內含電子郵件清單。



選用：放慢裝置動畫速度

由於本程式碼研究室涉及快速但精緻的轉場效果，因此放慢裝置動畫速度有助於觀察實作時轉場效果的細節。您可以使用 `adb` 殼層指令或「快速設定」方塊完成這項操作。請注意，這些減緩裝置動畫速度的方法也會影響 Reply 應用程式以外的裝置動畫。

方法 1：ADB Shell 指令

如要將裝置動畫速度減緩 10 倍，可以從指令列執行下列指令：


```
adb shell settings put global window_animation_scale 10
adb shell settings put global transition_animation_scale 10
adb shell settings put global animator_duration_scale 10
```

如要將裝置的動畫速度重設為正常，請執行下列指令：

```
adb shell settings put global window_animation_scale 1
adb shell settings put global transition_animation_scale 1
adb shell settings put global animator_duration_scale 1
```

方法 2：快速設定方塊

或者，如要設定快速設定方塊，請先在裝置上啟用開發人員設定 (如果先前未啟用)：

1. 開啟裝置的「設定」應用程式
2. 向下捲動至底部，然後按一下「關於模擬裝置」
3. 向下捲動至底部，快速點按「版本號碼」，直到啟用「開發人員設定」為止

接著，請在裝置的「設定」應用程式中執行下列操作，啟用「快速設定方塊」：

1. 按一下畫面頂端的搜尋圖示或搜尋列
2. 在搜尋欄位中輸入「動態磚」
3. 按一下「快速設定開發人員圖塊」列
4. 按一下「視窗動畫比例」切換鈕

最後，在整個程式碼研究室中，從螢幕頂端下拉系統通知欄，然後使用



圖示在慢速和正常速度動畫之間切換。

視裝置和 API 級別而定，上述設定快速設定方塊的指示可能略有不同，或不適用。

3. 熟悉範例應用程式程式碼

我們來看看程式碼。我們提供了一個應用程式，使用 [Jetpack 導覽元件](#) 程式庫在幾個不同的片段之間導覽，所有片段都位於單一活動 **MainActivity** 中：

- **HomeFragment**：顯示電子郵件清單
- **EmailFragment**：顯示單一完整電子郵件
- **ComposeFragment**：可撰寫新電子郵件
- **SearchFragment**：顯示搜尋檢視畫面

如要進一步瞭解本程式碼研究室中使用的相關概念，請參閱下列指南和練習：

- [開始使用 Android 適用的 Material 元件](#)
- [Android 進階功能 01.1：片段](#)
- [Kotlin Android 基礎知識 03.2：定義導覽路徑](#)
- [Kotlin Android 基礎知識 02.4：資料繫結基本概念](#)

navigation_graph.xml

首先，如要瞭解應用程式的導覽圖設定方式，請開啟 `app -> src -> main -> res -> navigation` 目錄中的 `navigation_graph.xml`：

```

<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/navigation_graph"
    app:startDestination="@id/homeFragment">

    <fragment
        android:id="@+id/homeFragment"
        android:name="com.materialstudies.reply.ui.home.HomeFragment"
        android:label="HomeFragment">
        <argument...>
        <action
            android:id="@+id/action_homeFragment_to_emailFragment"
            app:destination="@id/emailFragment" />
    </fragment>
    <fragment
        android:id="@+id/emailFragment"
        android:name="com.materialstudies.reply.ui.email.EmailFragment"
        android:label="EmailFragment">
        <argument...>
    </fragment>
    <fragment
        android:id="@+id/composeFragment"
        android:name="com.materialstudies.reply.ui.compose.ComposeFragment"
        android:label="ComposeFragment">
        <argument...>
    </fragment>
    <fragment
        android:id="@+id/searchFragment"
        android:name="com.materialstudies.reply.ui.search.SearchFragment"
        android:label="SearchFragment" />
    <action
        android:id="@+id/action_global_homeFragment"
        app:destination="@+id/homeFragment"
        app:launchSingleTop="true"
        app:popUpTo="@+id/navigation_graph"
        app:popUpToInclusive="true" />
    <action
        android:id="@+id/action_global_composeFragment"
        app:destination="@+id/composeFragment" />
    <action
        android:id="@+id/action_global_searchFragment"
        app:destination="@+id/searchFragment" />
</navigation>

```

請注意，上述所有片段都會顯示，且預設啟動片段會透過 `app:startDestination="@id/homeFragment"` 設為 `HomeFragment`。片段目的地圖的 XML 定義和動作，會告知您在連結轉場效果時遇到的 Kotlin 導覽程式碼。

activity_main.xml

接著，請查看 `app -> src -> main -> res -> layout` 目錄中的 `activity_main.xml` 版面配置。您會看到 `NavHostFragment`，其中已設定上述導覽圖：

```
<fragment
    android:id="@+id/nav_host_fragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:defaultNavHost="true"
    app:navGraph="@navigation/navigation_graph"/>
```

這個 `NavHostFragment` 會填滿畫面，並處理應用程式中的所有全螢幕片段導覽變更。`BottomAppBar` 和其錨定的 `FloatingActionButton` (同樣位於 `activity_main.xml` 中) 會顯示在 `NavHostFragment` 目前顯示的片段上方，因此會根據提供的範例應用程式程式碼，顯示或隱藏片段目的地。

此外，`activity_main.xml` 中的 `BottomNavDrawerFragment` 是底部導覽匣，內含可在不同電子郵件信箱之間導覽的選單，會透過「回覆」標誌按鈕 `BottomAppBar` 有條件地顯示。

MainActivity.kt

最後，如要查看導覽動作的使用範例，請開啟 `app -> src -> main -> java -> com.materialstudies.reply.ui` 目錄中的 `MainActivity.kt`。找出 `navigateToSearch()` 函式，該函式應如下所示：

```
private fun navigateToSearch() {
    val directions = SearchFragmentDirections.actionGlobalSearchFragment()
    findNavController(R.id.nav_host_fragment).navigate(directions)
}
```

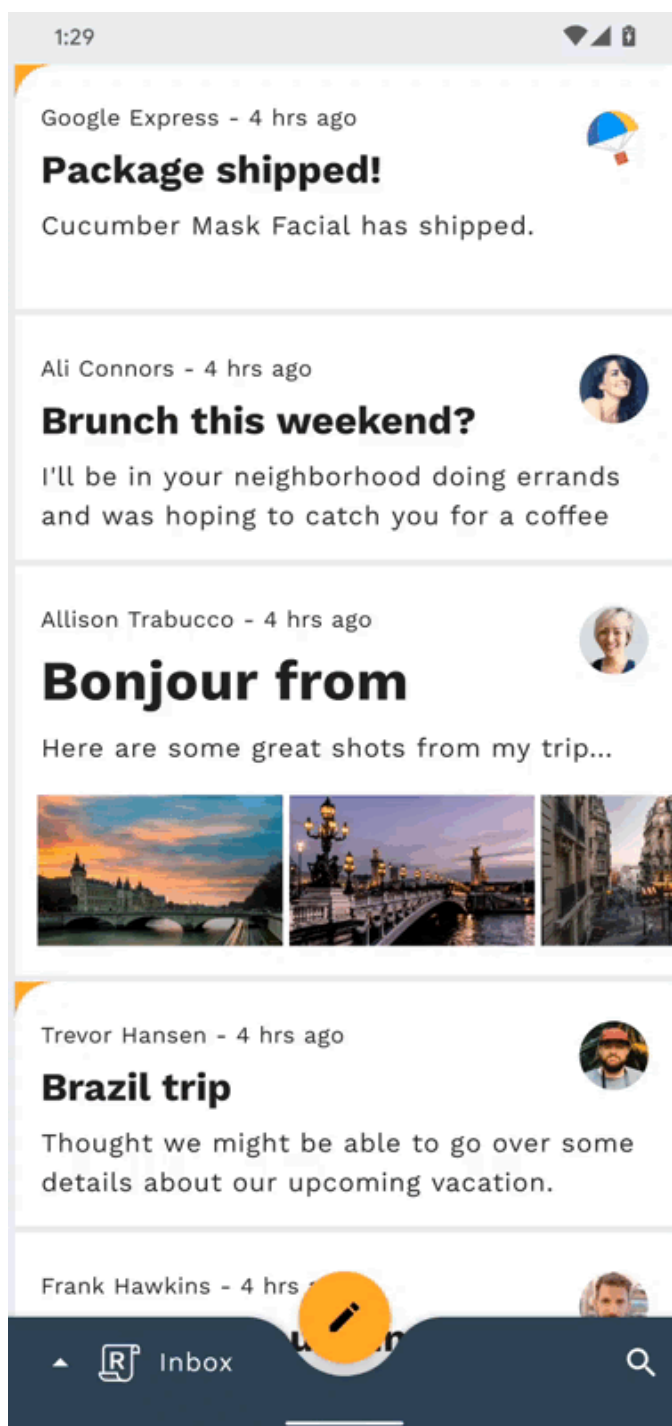
這會顯示如何前往搜尋檢視畫面頁面，且不使用任何自訂轉場效果。在本程式碼研究室中，您將深入瞭解 `Reply` 的 `MainActivity` 和四個主要 `Fragment`，設定與應用程式中各種 `Navigation` 動作搭配運作的 `Material` 轉場效果。

您已熟悉範例程式碼，接下來請實作第一個轉場效果。

4. 從電子郵件名單新增容器轉換轉場至電子郵件詳細資料頁面

首先，您要新增點選電子郵件時的轉場效果。容器轉換模式適用於這項導覽變更，因為這項模式是專為轉換含有容器的 UI 元素而設計。這個模式可在兩個 UI 元素之間建立起可見的連結。

加入任何程式碼前，請先嘗試執行 Reply 應用程式，然後點選電子郵件。這時應該會出現簡單的跳接，也就是畫面直接替換，沒有轉場效果：



MDC-Android 程式庫中的容器轉換稱為 `MaterialContainerTransform`。根據預設，這個 `Transition` 子類別會做為共用元素轉換運作，也就是說，當標示 `transitionName` 時，Android Transition 系統能夠在不同版面配置中挑選兩個檢視區塊。

首先，請在 `email_item_layout.xml` 的 `MaterialCardView` 中，於 `MaterialCardView` 上新增 `transitionName` 屬性，如以下程式碼片段所示：

email_item_layout.xml

```
android:transitionName="@{string/email_card_transition_name(email.id)}"
```

轉場效果名稱會採用含有參數的字串資源。您必須使用每封電子郵件的 ID，確保 `EmailFragment` 中的每個 `transitionName` 都是專屬的。

Reply 的最低 SDK 級別為 24，因此我們可以直接使用 API 級別 21 中導入的 `android:transitionName` 屬性。如果應用程式支援低於 API 級別 21 的版本，請在設定檢視區塊的轉場名稱時使用 `ViewCompat#setTransitionName` 方法。

您已設定電子郵件名單項目的轉場效果名稱，現在請在電子郵件詳細資料版面配置中執行相同操作。在 `fragment_email.xml` 中，將 `MaterialCardView` 的 `transitionName` 設為下列字串資源：

fragment_email.xml

```
android:transitionName="@string/email_card_detail_transition_name"
```

設定「開始」檢視區塊 (`email_item_layout.xml` 中的 `MaterialCardView`) 和「結束」檢視區塊 (`fragment_email.xml` 中的 `MaterialCardView`) 的轉換名稱，有助於將參與這項轉換的兩個共用元素檢視區塊告知 Android Transition 系統。

在 `HomeFragment.kt` 中，使用下列程式碼片段取代 `onEmailClicked` 中的程式碼，從開始檢視畫面 (電子郵件清單項目) 和結束檢視畫面 (電子郵件詳細資料畫面) 建立對應：

HomeFragment.kt

```
val emailCardDetailTransitionName = getString(R.string.email_card_detail_transition_name)
val extras = FragmentNavigatorExtras(cardView to emailCardDetailTransitionName)
val directions = HomeFragmentDirections.actionHomeFragmentToEmailFragment(email.id)
findNavController().navigate(directions, extras)
```

上述程式碼片段的重點是 `FragmentNavigatorExtras`。Android Transition 系統會取得在第一個參數 (`cardView`) 中設定的轉場效果名稱，並在該轉場效果名稱與做為第二個參數傳入的轉場效果名稱字串 (`emailCardDetailTransitionName`) 之間建立對應。系統會根據這項資訊找出要轉換的「開始」和「結束」檢視區塊，並提供給 `MaterialContainerTransform` 執行個體。

設定好管道後，您就可以建立容器轉換。在 `EmailFragment` `onCreate` 方法中，將 `sharedElementEnterTransition` 設為 `MaterialContainerTransform` 的新執行個體 (匯入

`com.google.android.material.transition` 版本，而非 `com.google.android.material.transition.platform` 版本)，方法是新增下列程式碼片段：

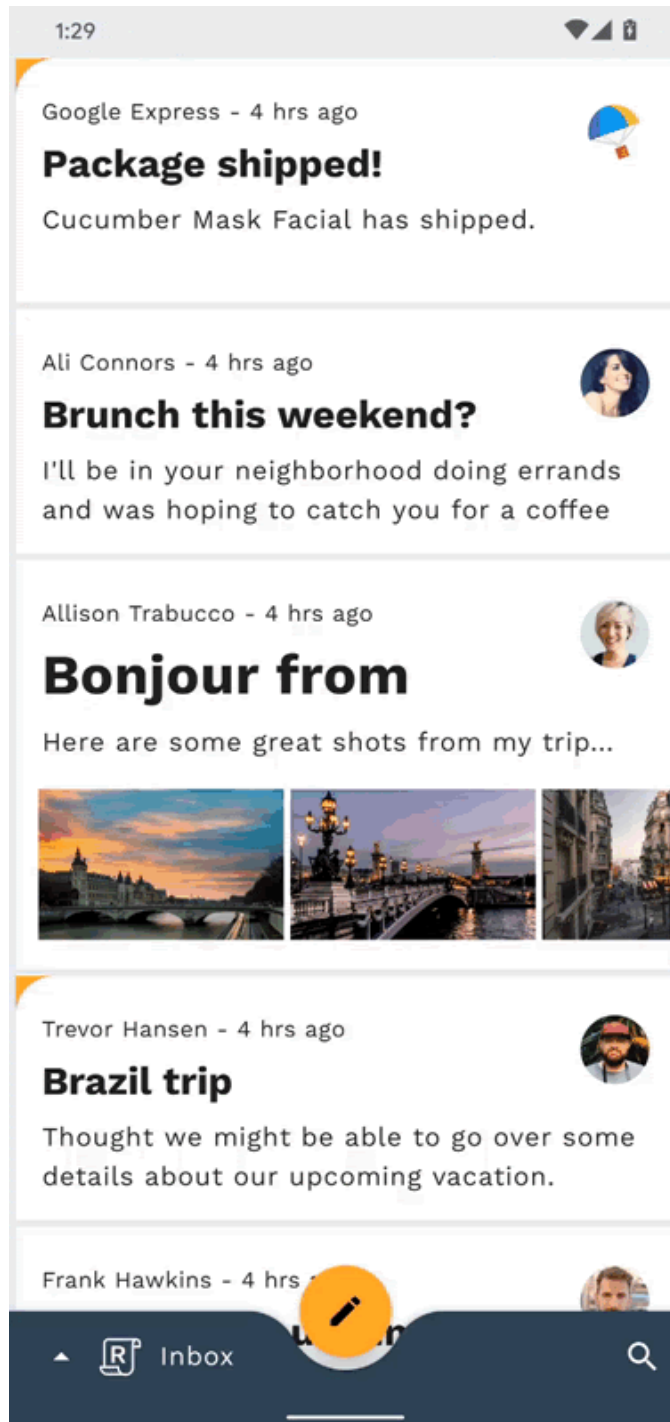
EmailFragment.kt

```
sharedElementEnterTransition = MaterialContainerTransform().apply {
    drawingViewId = R.id.nav_host_fragment
    duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
    scrimColor = Color.TRANSPARENT
    setAllContainerColors(requireContext().themeColor(R.attr.colorSurface))
}
```

這裡有幾點值得注意：

- 為求簡便，我們只會設定 `sharedElementEnterTransition`，而非 `sharedElementReturnTransition`。根據預設，如果沒有設定返回轉場效果，Android Transition 系統會在返回時自動反向執行進入轉場效果。
- `drawingViewId` 可控制動畫容器在檢視區塊階層中的放置位置。這可讓您在 UI 中其他元素下方或上方顯示轉場效果。在「回覆」的案例中，您會在與片段容器相同的層級執行容器轉換，確保轉換繪製在底部應用程式列和懸浮動作按鈕下方。
- `scrimColor` 是 `MaterialContainerTransform` 的屬性，可控制在動畫容器後方繪製的半透明陰影顏色。預設值為 32% 的黑色。這裡設為透明，表示不會繪製任何半透明幕簾。這是因為在後續步驟中，您會將結束轉場效果新增至電子郵件清單，與容器轉換效果完美搭配。
- 最後，我們將 `MaterialContainerTransform` 的容器顏色設為 `colorSurface`。當 `MaterialContainerTransform` 在兩個檢視區塊之間產生動畫時，會繪製到畫布的三個「容器」：1) 背景容器、2) 開始檢視區塊的容器，以及 3) 結束檢視區塊的容器。這三個容器都可以填滿顏色，預設為透明。如果開始或結束檢視區塊本身不會繪製背景，導致動畫期間會看到底下的其他元素，設定這些背景填滿顏色就很有用。由於這個範例中的所有容器都應設為 `colorSurface`，因此我們可以使用 `setAllContainerColors` 輔助方法，確保不會遇到任何視覺問題。

現在請嘗試重新執行應用程式。



一切看起來都很順利！當您點選電子郵件清單中的電子郵件時，容器轉換應將清單項目展開為全螢幕詳細資料頁面。不過請注意，按下返回鍵不會將電子郵件收合回清單。此外，電子郵件名單會在轉換開始時立即消失，並顯示灰色視窗背景。因此我們尚未完成。

通常，第一個問題 (摺疊功能無法運作) 是因為 Android Transition 系統嘗試執行返回轉場效果時，電子郵件清單尚未加載並填入 `RecyclerView`。我們需要一種方法，在 `HomeFragment` 列出清單之前等待。

Android Transition 系統提供 `postponeEnterTransition` 和 `startPostponedEnterTransition` 方法，可執行這項操作。如果呼叫 `postponeEnterTransition`，系統會暫緩執行任何進入轉換，直到呼叫 `startPostponedEnterTransition` 為止。這讓我們有機會「排定」轉移作業，直到 `RecyclerView` 填入電子郵件，且轉移作業能夠找到您設定的對應項為止。

如要修正返回轉場效果，請在 `HomeFragment.kt` 的 `onViewCreated` 方法中新增下列兩行：

HomeFragment.kt

```
postponeEnterTransition()  
view.doOnPreDraw { startPostponedEnterTransition() }
```

請嘗試重新執行應用程式。開啟電子郵件後按下返回鍵，電子郵件就會收合回清單。太好了！讓我們繼續改善動畫。

電子郵件名單消失的問題，是因為使用 `Navigation` 元件導覽至新的 `Fragment` 時，系統會立即移除目前的 `Fragment`，並以新的傳入 `Fragment` 取代。如要讓電子郵件名單在取代後仍可見，請為 `HomeFragment` 新增結束轉場效果。

`MDC-Android` 提供兩種轉場效果，可為您完成這項作業，分別是 `Hold` 和 `MaterialElevationScale`。`Hold` 轉場效果會將目標維持在目前位置，而 `MaterialElevationScale` 則會執行細微的縮放動畫。

將下列程式碼片段新增至 `HomeFragment` `onEmailClicked` 方法，即可在退出時讓電子郵件清單稍微縮放，並在重新進入時恢復原狀：

HomeFragment.kt

```
exitTransition = MaterialElevationScale(false).apply {  
    duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()  
}  
reenterTransition = MaterialElevationScale(true).apply {  
    duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()  
}
```

當目前的片段從返回堆疊中彈出後，片段就會重新進入檢視畫面。如果是 `HomeFragment`，從 `EmailFragment` 返回時會發生這種情況。在這裡，這會導致我們返回清單時，電子郵件清單會稍微放大。

接著，為了確保 `MaterialElevationScale` 轉換套用至整個主畫面，而非階層中的每個個別檢視區塊，請將 `fragment_home.xml` 中的 `RecyclerView` 標示為轉換群組。

fragment_home.xml

```
android:transitionGroup="true"
```

`Reply` 的最低 SDK 級別為 24，因此我們可以直接使用 API 級別 21 中導入的 `android:transitionGroup` 屬性。如果應用程式支援低於 API 級別 21 的版本，請在設定檢視區塊的轉換群組時使用 `ViewGroupCompat#setTransitionGroup` 方法。

此時，您應該已擁有可正常運作的容器轉換。點選電子郵件後，清單項目會展開為詳細資料畫面，電子郵件清單則會縮小。按下返回鍵會將電子郵件詳細資料畫面收合回清單項目，同時放大電子郵件清單。

1:29



Google Express - 4 hrs ago



Package shipped!

Cucumber Mask Facial has shipped.

Ali Connors - 4 hrs ago



Brunch this weekend?

I'll be in your neighborhood doing errands and was hoping to catch you for a coffee

Allison Trabucco - 4 hrs ago



Bonjour from

Here are some great shots from my trip...



Trevor Hansen - 4 hrs ago



Brazil trip

Thought we might be able to go over some details about our upcoming vacation.

Frank Hawkins - 4 hrs ago

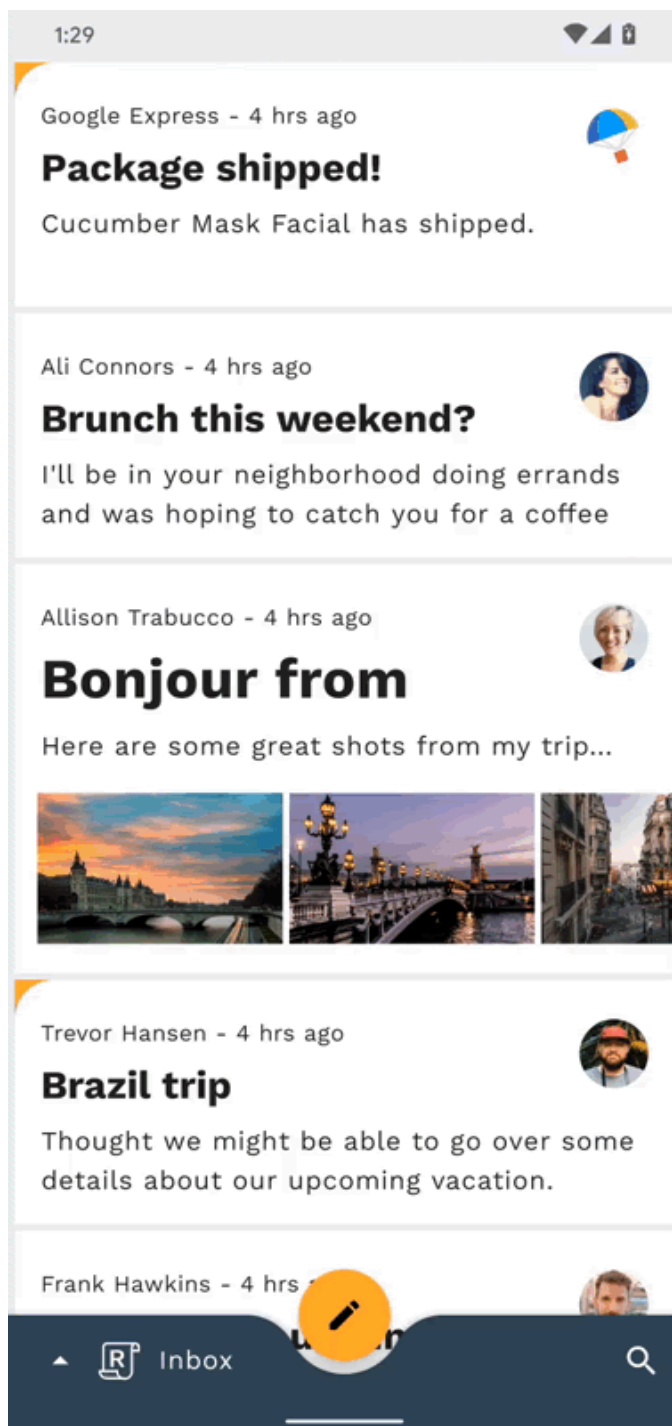


Inbox



5. 從 FAB 新增容器轉換轉場效果至撰寫電子郵件頁面

我們將繼續使用容器轉換，並從懸浮動作按鈕新增至 `ComposeFragment` 的轉場效果，將懸浮動作按鈕展開為使用者要撰寫的新電子郵件。首先，請重新執行應用程式，然後點選懸浮動作按鈕 (FAB)，確認啟動電子郵件撰寫畫面時沒有轉場效果。



如要查看上述跳轉效果，請在重新啟動應用程式後，查看電子郵件名單時點選懸浮動作按鈕 (FAB)。如果您在目前執行應用程式期間開啟過電子郵件，可能會因為過時的結束或重新進入轉場效果，而看到部分轉場效果。如果您使用 Android Studio 的熱重載執行應用程式，就特別容易發生這種情況。完成本程式碼研究室的所有流程後，就不會再有問題。

雖然我們使用相同的轉場效果類別，但設定這個執行個體的方式會有所不同，因為懸浮動作按鈕 (FAB) 位於 `MainActivity` 中，而 `ComposeFragment` 則放在 `MainActivity` Navigation 主機容器內。

在 `ComposeFragment.kt` 中，將下列程式碼片段加到 `onViewCreated` 方法，請務必匯入 `Slide` 的 `androidx.transition` 版本。

ComposeFragment.kt

```
enterTransition = MaterialContainerTransform().apply {
    startView = requireActivity().findViewById(R.id.fab)
    endView = emailCardView
    duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
    scrimColor = Color.TRANSPARENT
    containerColor = requireContext().themeColor(R.attr.colorSurface)
    startContainerColor = requireContext().themeColor(R.attr.colorSecondary)
    endContainerColor = requireContext().themeColor(R.attr.colorSurface)
}
returnTransition = Slide().apply {
    duration = resources.getInteger(R.integer.reply_motion_duration_medium).toLong()
    addTarget(R.id.email_card_view)
}
```

除了用於設定先前容器轉換的參數外，這裡還會手動設定 `startView` 和 `endView`。您不必使用 `transitionName` 屬性讓 Android Transition 系統瞭解要轉換哪些檢視區塊，而是可以在必要時手動指定這些檢視區塊。

現在請重新執行應用程式，您應該會看到懸浮動作按鈕 (FAB) 轉換為撰寫畫面 (請參閱本步驟結尾的 GIF)。

與上一個步驟類似，您需要為 `HomeFragment` 新增轉場效果，避免在移除並由 `ComposeFragment` 取代後消失。

將下列程式碼片段複製到 `MainActivity` 的 `navigateToCompose` 方法中，然後呼叫 `NavController` `navigate`。

MainActivity.kt

```
currentNavigationFragment?.apply {
    exitTransition = MaterialElevationScale(false).apply {
        duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
    }
    reenterTransition = MaterialElevationScale(true).apply {
        duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
    }
}
```

請注意，`MaterialElevationScale` 的時間長度與 `MaterialContainerTransform` 相同。這可確保 `HomeFragment` 的保留時間與容器轉換的執行時間相同，且容器轉換完成後就會立即消失。

這個步驟就完成了！您應該會看到從 FAB 到 Compose 畫面的轉場效果，如下所示：

1:29



Google Express - 4 hrs ago



Package shipped!

Cucumber Mask Facial has shipped.

Ali Connors - 4 hrs ago



Brunch this weekend?

I'll be in your neighborhood doing errands and was hoping to catch you for a coffee

Allison Trabucco - 4 hrs ago



Bonjour from

Here are some great shots from my trip...



Trevor Hansen - 4 hrs ago



Brazil trip

Thought we might be able to go over some details about our upcoming vacation.

Frank Hawkins - 4 hrs ago



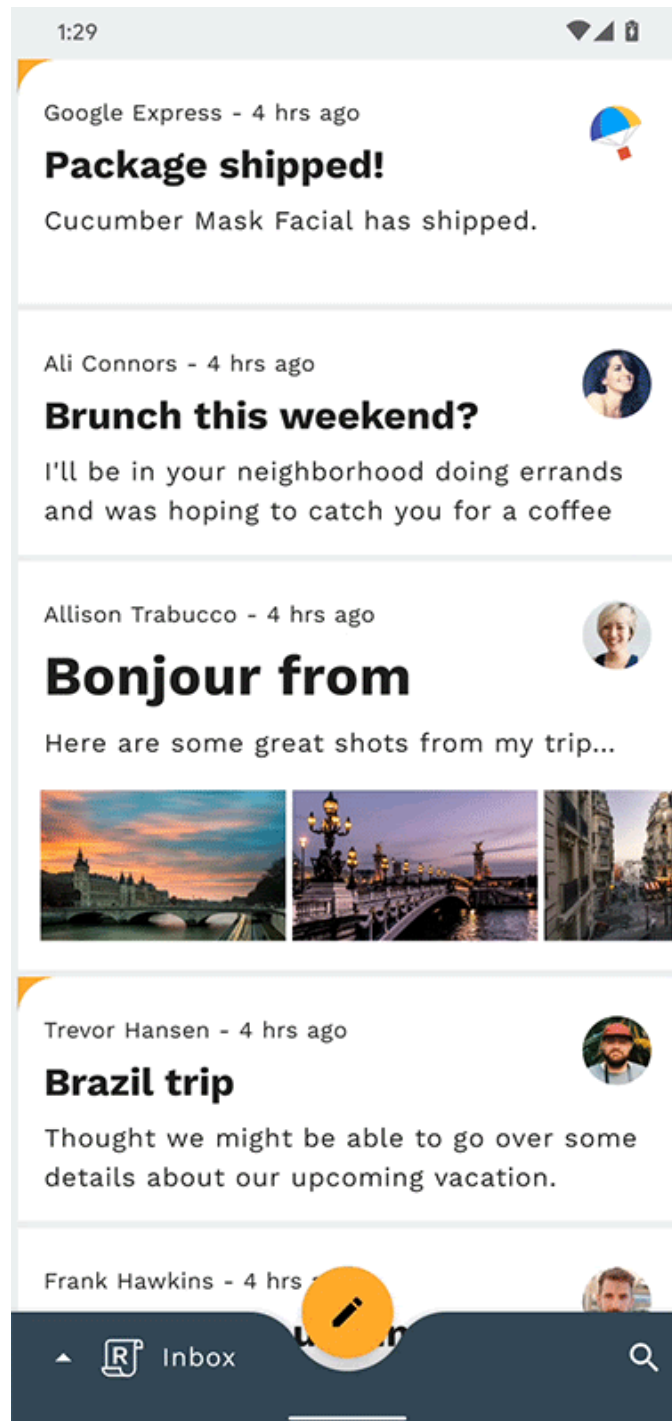
Inbox



6. 從搜尋圖示到搜尋檢視畫面頁面，新增共用 Z 軸轉換

在這個步驟中，我們會新增從搜尋圖示到全螢幕搜尋檢視區塊的轉場效果。由於這項導覽變更不涉及任何持續性容器，我們可以運用「共用 Z 軸」轉場效果，強化兩個畫面之間的空間關係，並指出應用程式階層向上移動一個層級。

新增任何其他程式碼前，請先嘗試執行應用程式，然後輕觸畫面右下角的搜尋圖示。這時應該會顯示搜尋檢視畫面，且沒有任何轉場效果。



首先，在 `MainActivity` 中找出 `navigateToSearch` 方法，並在 `NavController` `navigate` 方法呼叫前新增下列程式碼片段，設定目前片段的結束和重新進入 `MaterialSharedAxis` Z 軸轉場。

`MainActivity.kt`

```
currentNavigationFragment?.apply {
    exitTransition = MaterialSharedAxis(MaterialSharedAxis.Z, true).apply {
        duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
    }
    reenterTransition = MaterialSharedAxis(MaterialSharedAxis.Z, false).apply {
        duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
    }
}
```

接著，在 `SearchFragment` 的 `onCreate` 方法中新增下列程式碼片段，設定進入和返回 `MaterialSharedAxis` 轉場效果。

SearchFragment.kt

```
enterTransition = MaterialSharedAxis(MaterialSharedAxis.Z, true).apply {
    duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
}
returnTransition = MaterialSharedAxis(MaterialSharedAxis.Z, false).apply {
    duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()
}
```

請注意，這兩個片段會一起移動，與 `MaterialSharedAxis` `forward` 布林建構函式參數有關。當 `MainActivity` 目前的片段結束時，`SearchFragment` 會進入。因此，在 `MainActivity` 目前的片段中，結束轉換是向前，在 `SearchFragment` 進入轉換也是向前。這樣可確保這兩項片段在執行這些轉場效果配對時，會朝相同方向移動。

反方向則相反。當 `SearchFragment` 結束時，`MainActivity` 目前的片段會進入。因此，`SearchFragment` 會設為向後傳回，而 `MainActivity` 目前的片段會設為向後重新進入。

最後，為確保 `MaterialSharedAxis` 轉場效果套用至整個搜尋畫面，而非階層中的每個個別檢視區塊，請將 `fragment_search.xml` 中的 `LinearLayout` 標示為轉場群組。

fragment_search.xml

```
android:transitionGroup="true"
```

`Reply` 的最低 SDK 級別為 24，因此我們可以直接使用 API 級別 21 中導入的 `android:transitionGroup` 屬性。如果應用程式支援低於 API 級別 21 的版本，請在設定檢視區塊的轉換群組時使用 `ViewGroupCompat#setTransitionGroup` 方法。

大功告成！現在請嘗試重新執行應用程式，然後輕觸搜尋圖示。主畫面和搜尋畫面應同時淡出，並沿著 Z 軸縮放，在兩個畫面之間營造流暢效果。

1:29



Google Express - 4 hrs ago



Package shipped!

Cucumber Mask Facial has shipped.

Ali Connors - 4 hrs ago



Brunch this weekend?

I'll be in your neighborhood doing errands and was hoping to catch you for a coffee

Allison Trabucco - 4 hrs ago



Bonjour from

Here are some great shots from my trip...



Trevor Hansen - 4 hrs ago



Brazil trip

Thought we might be able to go over some details about our upcoming vacation.

Frank Hawkins - 4 hrs ago



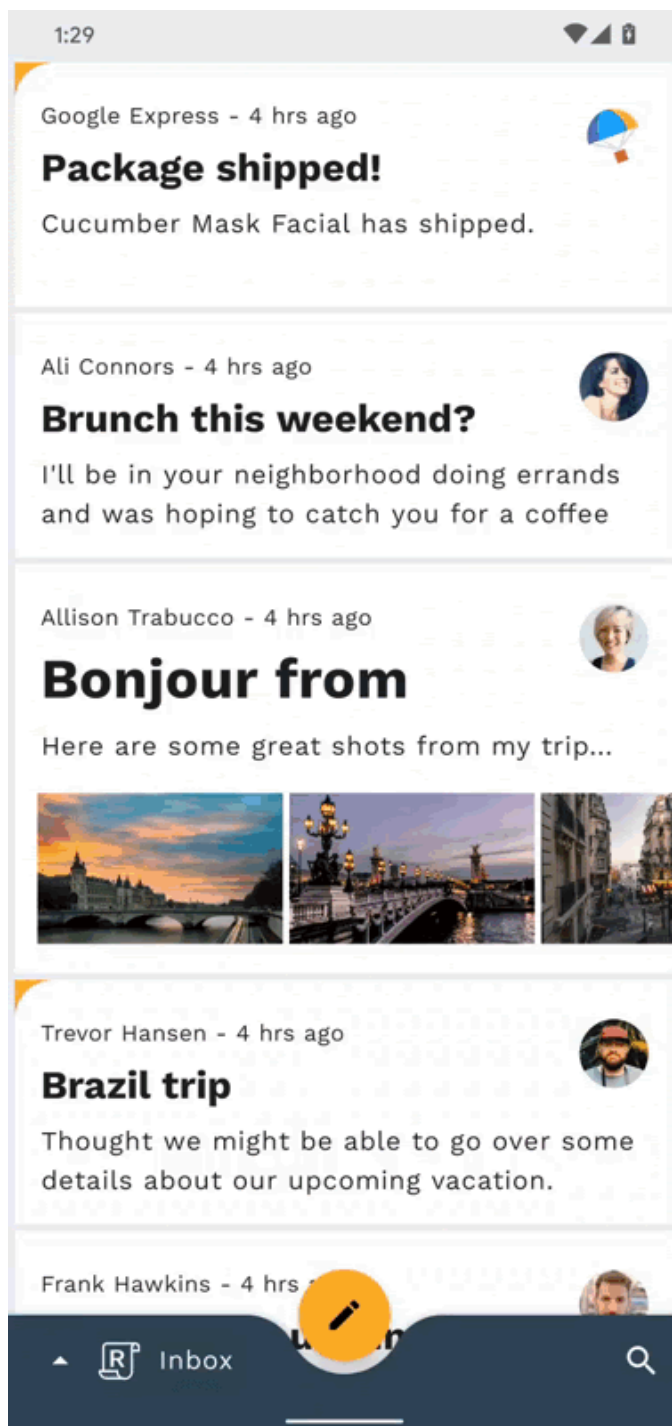
▲  Inbox



7. 在信箱頁面之間新增「淡出淡入」轉場效果

在這個步驟中，我們會在不同信箱之間新增轉場效果。由於我們不想強調空間或階層關係，因此會使用淡入淡出效果，在電子郵件清單之間執行簡單的「交換」作業。

在新增任何其他程式碼之前，請嘗試執行應用程式、輕觸底部應用程式列中的 Reply 標誌，然後切換信箱。電子郵件清單應會變更，且不會有轉場效果。



首先，在 `MainActivity` 中找出 `navigateToHome` 方法，並在 `NavController` `navigate` 方法呼叫之前新增下列程式碼片段，設定目前片段的結束 `MaterialFadeThrough` 轉場。

MainActivity.kt

```
currentNavigationFragment?.apply {  
    exitTransition = MaterialFadeThrough().apply {  
        duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()  
    }  
}
```

接著開啟 `HomeFragment`。在 `onCreate` 中，將片段的 `enterTransition` 設為 `MaterialFadeThrough` 的新執行個體。

HomeFragment.kt

```
enterTransition = MaterialFadeThrough().apply {  
    duration = resources.getInteger(R.integer.reply_motion_duration_large).toLong()  
}
```

由於信箱的運作方式不同，上述設定與先前的步驟略有差異。信箱只是 `HomeFragment`，加上一個引數，可告知片段要顯示的電子郵件清單。此外，由於 `HomeFragment` 在導覽圖中設為 `singleTop=true`，因此您一律會*向前*導覽至新信箱，不必處理返回或重新輸入的轉場效果 (在彈出片段時執行的轉場效果)。因此，您只需要設定 `HomeFragment` 進入和離開轉換。

這是架構情境的範例，會影響轉場效果的設定方式。在自己的應用程式中新增轉場效果時，您也需要考量類似事項！

重新執行應用程式。開啟底部導覽匣並變更信箱時，目前的電子郵件清單應會淡出並縮放，而新清單則會淡入並縮放。太棒了！

1:29



Google Express - 4 hrs ago



Package shipped!

Cucumber Mask Facial has shipped.

Ali Connors - 4 hrs ago



Brunch this weekend?

I'll be in your neighborhood doing errands and was hoping to catch you for a coffee

Allison Trabucco - 4 hrs ago



Bonjour from

Here are some great shots from my trip...



Trevor Hansen - 4 hrs ago



Brazil trip

Thought we might be able to go over some details about our upcoming vacation.

Frank Hawkins - 4 hrs ago



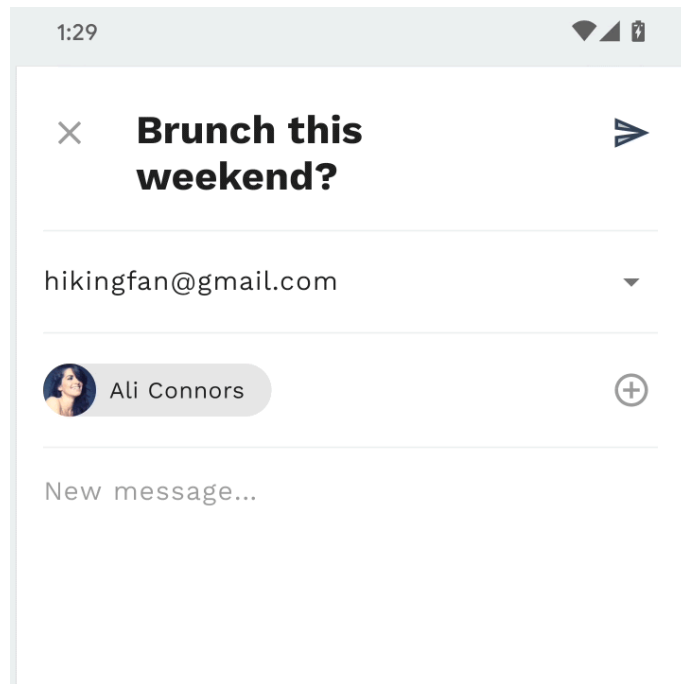
▲  Inbox



8. 從電子郵件地址方塊到資訊卡檢視畫面，新增容器轉換轉場效果

在這個步驟中，您要新增轉場效果，將方塊轉換為彈出式資訊卡。這裡使用容器轉換，是為了讓使用者瞭解在彈出式視窗中執行的動作，會影響彈出式視窗的來源晶片。

新增任何程式碼前，請先執行 Reply 應用程式，點選電子郵件，然後點選「回覆」FAB，並嘗試點選收件者的聯絡人晶片。晶片應會立即消失，且系統會顯示含有該聯絡人電子郵件地址的資訊卡，不會有任何動畫。



您將在 `ComposeFragment` 中完成這個步驟。`ComposeFragment` 版面配置中已新增收件者方塊 (預設為顯示) 和收件者資訊卡 (預設為隱藏)。收件者晶片和這張資訊卡是您要建立容器轉換的兩個檢視畫面。

首先，請開啟 `ComposeFragment` 並找到 `expandChip` 方法。點選提供的 `chip` 時，系統會呼叫這個方法。在交換 `recipientCardView` 和 `chip` 可見度的行上方，新增下列程式碼片段，這會觸發透過 `beginDelayedTransition` 註冊的容器轉換。

ComposeFragment.kt

```
val transform = MaterialContainerTransform().apply {
    startView = chip
    endView = binding.recipientCardView
    scrimColor = Color.TRANSPARENT
    endElevation = requireContext().resources.getDimension(
        R.dimen.email_recipient_card_popup_elevation_compat
    )
    addTarget(binding.recipientCardView)
}

TransitionManager.beginDelayedTransition(binding.composeConstraintLayout, transform)
```

上述轉換設定的重要部分是呼叫 `addTarget`。根據預設，`TransitionManager` 會在所有變更的檢視區塊上執行提供的轉場效果。設定目標可確保容器轉換只會在指定的單一目標檢視畫面中執行。

如果您現在執行應用程式，晶片應會轉換為收件者的電子郵件地址卡片。接著，我們來設定返回轉場效果，將資訊卡收合回晶片。

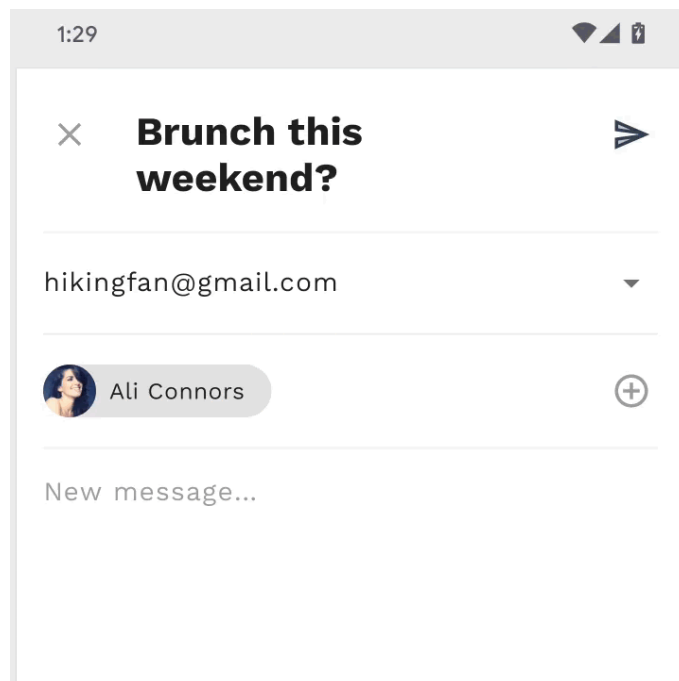
在 `ComposeFragment` 的 `collapseChip` 方法中，新增下列程式碼片段，將資訊卡收合回晶片。

ComposeFragment.kt

```
val transform = MaterialContainerTransform().apply {
    startView = binding.recipientCardView
    endView = chip
    scrimColor = Color.TRANSPARENT
    startElevation = requireContext().resources.getDimension(
        R.dimen.email_recipient_card_popup_elevation_compat
    )
    addTarget(chip)
}

TransitionManager.beginDelayedTransition(binding.composeConstraintLayout, transform)
```

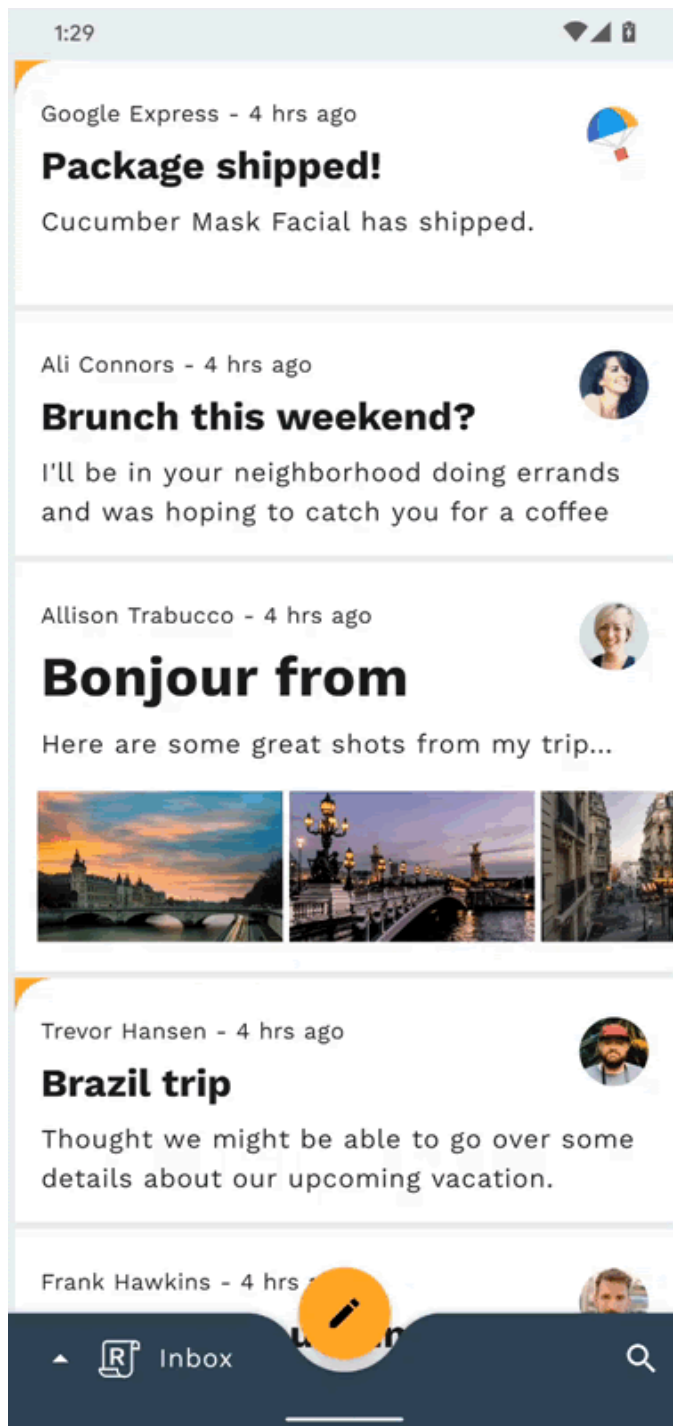
重新執行應用程式。按一下晶片應會將晶片展開為資訊卡，而按一下資訊卡則會將資訊卡收合回晶片。太棒了！



步驟 9 · 約 0 分鐘

9. 大功告成

您只用了不到 100 行的 Kotlin 程式碼和一些基本 XML 標記，MDC-Android 程式庫就協助您在符合 Material Design 指南的現有應用程式中，建立精美的轉場效果，而且在所有 Android 裝置上看起來和運作方式都一致。



如要取得已完成的 Reply Material 動態系統應用程式，請下載[已完成的程式碼研究室](#)，或使用 Git 查看 `complete` 分支版本：`git checkout complete`。

如要查看本程式碼研究室的所有程式碼變更，請參閱 `complete` 分支版本的[差異比較](#)。

後續步驟

如要進一步瞭解 Material 動態效果系統，請務必參閱[規格](#)和完整[開發人員說明文件](#)，並嘗試在應用程式中加入一些 Material 轉場效果！

感謝您試用 Material 動作。希望您喜歡這個程式碼研究室！
